Heart Disease Prediction Using Machine Learning Techniques on Structured Electronic Health Records (EHRs)

Name of Student:

Department:

Registration No.:

Name of Research Supervisor:

Abstract

This dissertation discusses the supervised machine learning to identify heart disease at an earlier stage using structured Electronic Health Records (EHRs). The research targets the use of Logistic regression, Support Vector Machine (SVM) and random forest classifiers, which are trained using publicly available Kaggle datasets of demographic and clinical and lifestyle variables (age, cholesterol, blood pressure, chest pain, exercise induced angina, and ST-slope) on these variables. The methodology combines the process of systematic data acquisition, preprocessing, training, and evaluation, and it is then explained by SHAP and LIME to make clinical transparency more transparent. Precision, recall, accuracy, F1-score, ROC and Precision-Recall curves were used to measure model performance. Findings showed that all the three models were highly predictive with SVM having the best ROC-AUC. The age, cholesterol levels, the type of chest pain and ST-slope were found to be critical predictors through the feature importance analysis. Provided explainable AI-based methods offer interpretable insights, which are consistent with the statistical results and consistent with clinical reasoning, ethical needs of transparency and trust in healthcare AI can be met. The results verify that machine learning models, with the support of interpretable frameworks may be used as useful instruments in risk stratification and decision support in cardiology. Practical implications emphasize the possible inclusion into clinical working processes, whereas limitations, including data set size, representativeness, and computing constraints, imply the need to exercise caution in the generalization. Future work recommendations will also be to increase different datasets, experiment with explainable deep architectures, and use federated learning to implement privacy-aware, scalable healthcare solutions. On the whole, the paper shows that explainable machine learning pipelines are ethically and effectively valuable to provide proactive cardiovascular care.

Acknowledgement

I would like to greatly appreciate my supervisor who was of great assistance through guidance, constructive feedback, and encouragement during this research. I would also like to thank my department with academic support and the Kaggle community because of this study through the open-access datasets.

Table of Contents

Abstract 2

Acknowledgement 3

Chapter 1: Introduction 10

1.1 Project Summary 10

1.2 Aim: 10

1.3 Objective: 10

1.4 Research questions: 10

1.5 Problem Statement: 11

1.6 Research Area – A Brief Literature Review 11

1.7 Expected Practical Element Output 13

1.8 Required Resources 14

1.9 Prerequisite Knowledge/Skills Required 14

1.10 Project Plan – Gantt Chart 15

Chapter 2: Literature Review/Related Work 16

2.1 Comprehensive Overview of the Existing Literature 16

2.1.1 Introduction to Heart Disease and Its Prediction Challenges 16

2.1.2 Evolution of Machine Learning in Healthcare 17

2.1.3 Importance and Potential of Electronic Health Records (EHRs) 18

2.2 Critical Analysis of Existing Studies 19

2.2.1 Traditional Datasets and Their Limitations 19

2.2.2 Advancements Through Structured EHR Data 20

2.2.3 Comparison of Machine Learning Techniques 22

2.2.4 Role of Feature Engineering and Selection 23

2.2.5 Evaluation Metrics Used in Prediction Models 24

2.2.6 Interpretability and Clinical Relevance 24
2.3 Research Gaps and Emerging Trends 25
2.3.1 Data Quality Gap in Data and Preprocessing 25
2.3.2 Generalizability Across Populations 27
2.3.3 Integration of Multimodal Data 28
2.3.4 Real-Time and Dynamic Prediction 28
2.3.5 Privacy and Ethical Considerations 29
2.4 Summary and Future Research Directions 29
2.4.1 Summary of Literature Review 30
2.4.2 Future Developments 30
Chapter 3: Methodology 32
3.1 Introduction 32
3.2 Research Design 32
3.2.1 Quantitative Experimental Approach 32
3.2.2 Controlled Algorithm Comparison 33
3.3 Dataset Selection and Description 33
3.3.1 Source and Access 33
3.3.2 Structure and Features 33
3.4 Data Preprocessing 35
3.4.1 Data Cleaning 35
3.4.2 Encoding of Categorical Variables 35
3.5 Model Development 36
3.5.1 Logistic Regression 36
3.5.2 Support Vector Machine 36
3.5.3 Random Forest 36

3.6 Model Evaluation 37
3.6.1 Evaluation Metrics 37
3.6.2 Validation Strategy 37
3.7 Explainable AI Integration 38
3.8 Ethical Considerations 38
3.9 Conclusion 38
Chapter 4: Implementation and Development 40
4.1 Environment Setup 40
4.1.1 Installation of Dependencies 40
4.1.2 Rationale for Package Selection 41
4.1.3 Verification of Setup 41
4.2 Dataset Acquisition 41
4.3 Exploratory Data Analysis 42
4.3.1 Distribution of Numerical Features 42
4.3.2 Distribution of Categorical Features 43
4.3.3 Correlation Heatmap 43
4.3.4 Target Distribution 44
4.4 Data Preprocessing 45
4.4.1 Feature Separation 45
4.4.2 Train-Test Split 45
4.4.3 Column Transformer Design 46
4.4.4 The Preprocessing significance 47
4.5 Model Development 47
4.5.1 Logistic Regression 47
4.5.2 Support Vector Machine 47

4.5.3 Random Forest Classifier 48
4.5.4 Training Strategy 48
4.6 Model Evaluation 49
4.6.1 Evaluation Metrics 49
4.6.2 Visualization of Metrics 50
4.6.3 Confusion Matrices 50
4.6.4 ROC and Precision-Recall Curves 51
4.7 Model Explainability 51
4.7.1 SHAP Integration 51
4.7.2 SHAP for Support Vector Machine 52
4.7.3 SHAP for Random Forest and Logistic Regression 52
4.7.4 SHAP Waterfall Plots 53
4.8 Feature Importance 54
4.9 Conclusion 55
Chapter 5: Results and Evaluation 56
5.1 Model Training and Evaluation 56
5.1.1 Logistic Regression Performance 56
5.1.2 Support Vector Machine Performance 57
5.1.3 Random Forest Performance 57
5.2 Comparative Evaluation of Models 58
5.2.1 Confusion Matrices 59
5.2.2 ROC and Precision-Recall Curves 60
5.2.3 Metric Comparisons via Bar Plots 62
5.3 Explainability and Interpretability 63
5.3.1 SHAP Summary Results 63

5.3.2 SHAP Waterfall Results 64
5.3.3 LIME Explanations 65
5.4 Feature Importance Analysis 66
5.5 Linking Results to Research Objectives 67
5.6 Addressing Research Questions 67
5.7 Overall Evaluation 68
5.8 Conclusion 68
Chapter 6: Conclusion and Recommendation 70
6.1 Introduction 70
6.2 Justification of Aim and Objectives 70
6.3 Key Findings 70
6.4 Contribution to Knowledge 71
6.5 Practical Implications 71
6.6 Limitations of the Study 72
6.7 Recommendations for Future Work 72
6.8 Final Remarks 72
References 74
Appendix: Code Section and Dataset 85

List of Figures

Figure 1 Installation of Dependencies 40
Figure 2 Data Downloading from Kaggle 42
Figure 3 Numeric Distribution plot implementation 43
Figure 4 Categorical Count Implementation 43

Figure 5 Correlation Heatmap 44
Figure 6 Target Distribution 45
Figure 7 Feature Separation Implementation 45
Figure 8 Model Training Implementation 46
Figure 9 Model Training code 49
Figure 10 Model Evaluation Code 50
Figure 11 Shap Integration 52
Figure 12 SHAP for Support Vector Machine 52
Figure 13 SHAP for Random Forest and Logistic Regression 53
Figure 14 SHAP Waterfall Plots 54
Figure 15 Feature Importance 55
Figure 16 Logistic Regression Result 57
Figure 17 Support Vector Machine result 57
Figure 18 Random Forest Result 58
Figure 19 Comparative Evaluation Result 59
Figure 20 Logistic Regression Confusion matrix 60
Figure 21 SVM Confusion matrix 60
Figure 22 RF Confusion matrix 60
Figure 23 LogReg ROC 61
Figure 24 SVM ROC 61
Figure 25 RF ROC 62
Figure 26 Metric Comparisons via Bar Plots ROC AUC 62
Figure 27 F! Comparison Plots 63
Figure 28 SHAP Result 64
Figure 29 SHAP Waterfall Results 65
Figure 30 LIME Explanations result 66
Figure 31 Feature Importance Analysis Result 67

Chapter 1: Introduction

1.1 Project Summary

In this project, a machine learning-based tool to predict heart disease on the basis of the structured Electronic Health Records (EHRs) will be developed. It will aim at using supervised learning algorithms (Logistic Regression, Support Vector Machine, Random Forest) and training them on publicly available datasets (e.g. Kaggle) with age, blood pressure, cholesterol, and heart rate features. Accuracy, precision, recall and ROC-AUC will be used to assess the performance of the model (Mandair et al., 2020). Tools that are explainable AI (XAI) will be provided so that models are transparent. It will allow ethical and evidence-based clinical choices by the use of a Python-based implementation.

1.2 Aim:

To build and test the predictive system based on machine learning to predict early signs of heart disease by a structured Electronic Health Records to support open and ethical clinical decision-making.

1.3 Objective:

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

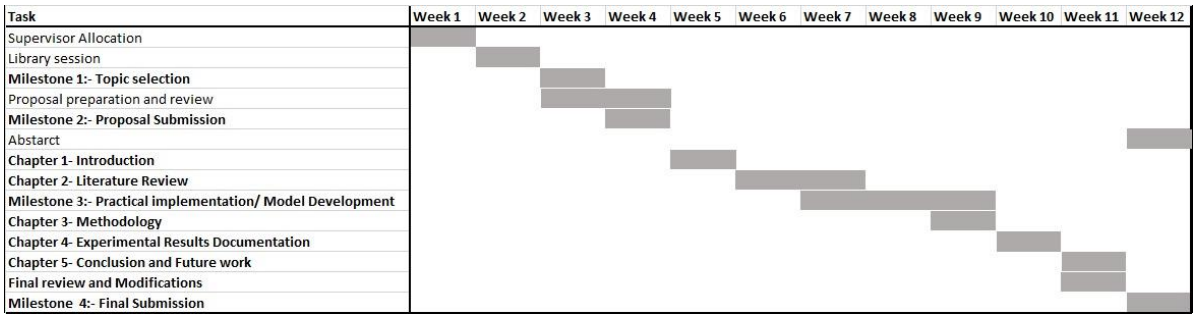
[redacted]

[redacted]

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

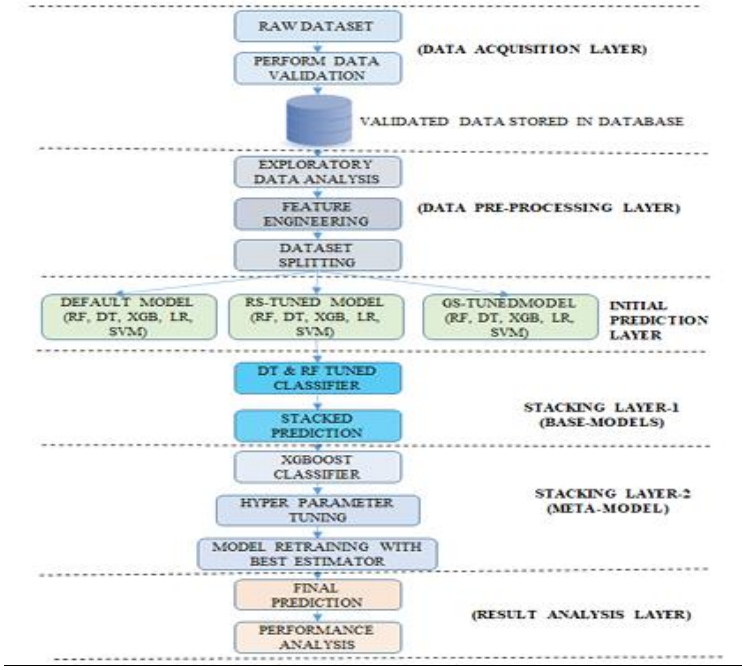
[redacted]

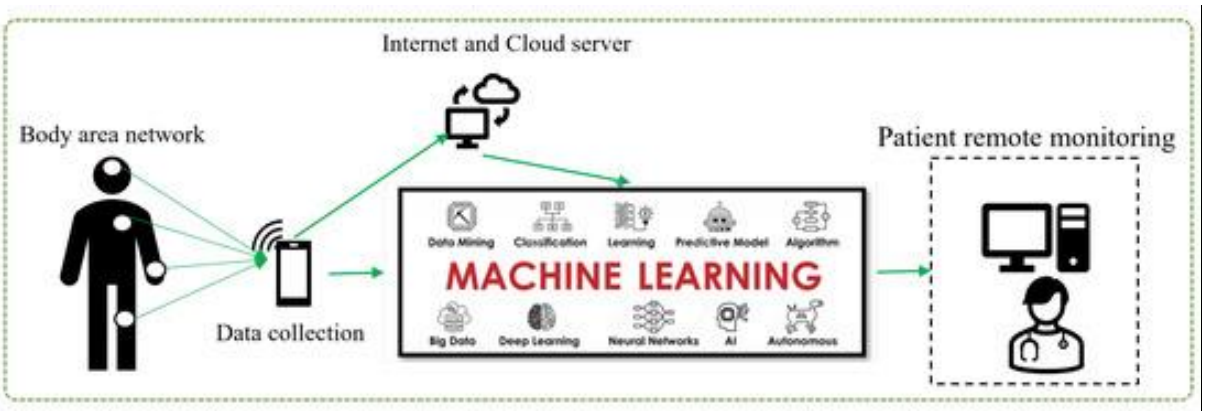
[redacted]

[redacted]

[redacted]

[redacted]


RAW DATASET
PERFORM DATA VALIDATION
(DATA ACQUISITION LAYER)
VALIDATED DATA STORED IN DATABASE
EXPLORATORY DATA ANALYSIS
FEATURE ENGINEERING
(DATA PRE-PROCESSING LAYER)
DATASET SPLITTING
DEFAULT MODEL (RF, DT, XGB, LR, SVM)
RS-TUNED MODEL (RF, DT, XGB, LR, SVM)
GS-TUNEDMODEL (RF, DT, XGB, LR, SVM)
INITIAL PREDICTION LAYER
DT & RF TUNED CLASSIFIER
STACKED PREDICTION
STACKING LAYER-1 (BASE-MODELS)
XGBOOST CLASSIFIER
HYPER PARAMETER TUNING
MODEL RETRAINING WITH BEST ESTIMATOR
STACKING LAYER-2 (META-MODEL)
FINAL PREDICTION
PERFORMANCE ANALYSIS
(RESULT ANALYSIS LAYER)


Internet and Cloud server
Body area network
Patient remote monitoring
Data collection
Data Mining
Classification
Learning
Predictive Model
Algorithm
MACHINE LEARNING
Big Data
Deep Learning
Neural Networks
AI
Autonomous

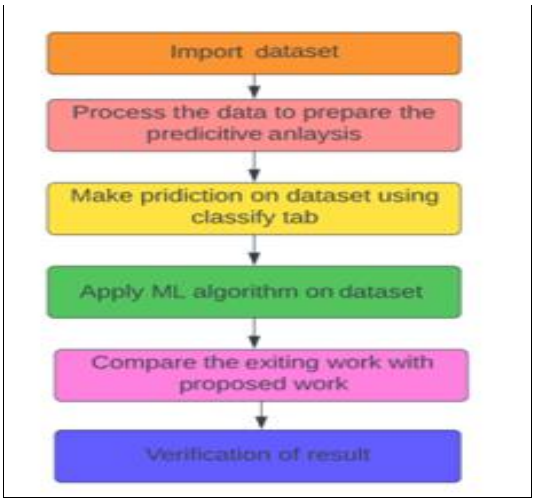
[redacted]

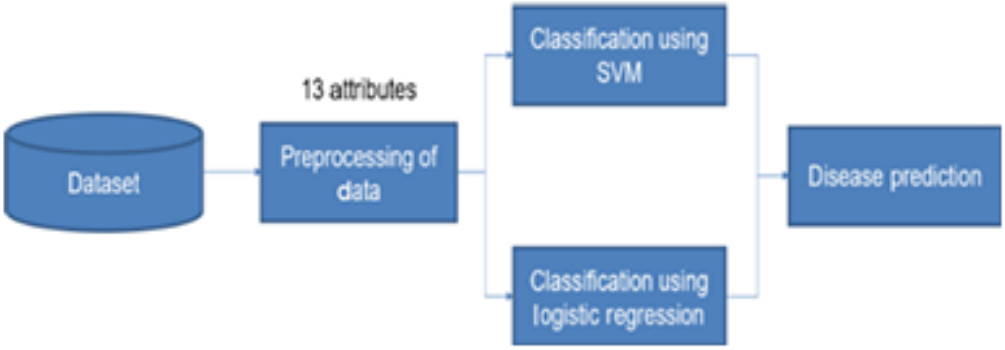
[redacted]

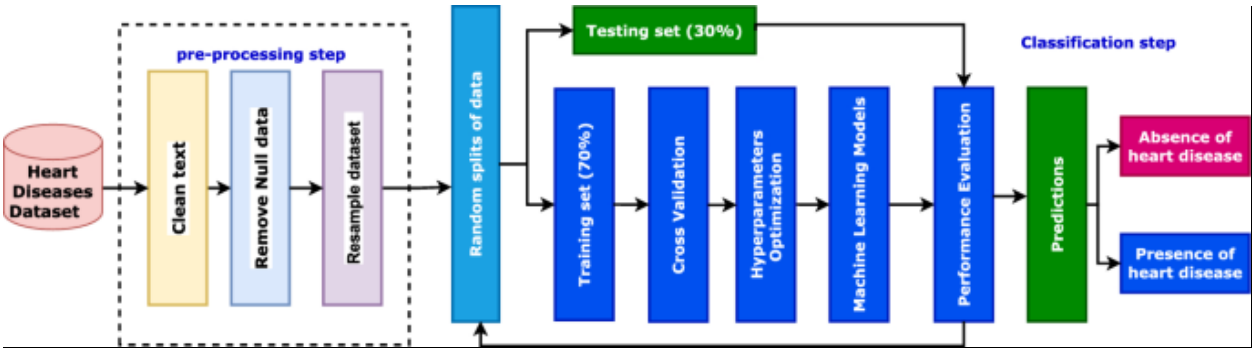
[redacted]

[redacted]

[redacted]


Import dataset
Process the data to prepare the predicitive anlaysis
Make pridiction on dataset using classify tab
Apply ML algorithm on dataset
Compare the exiting work with proposed work
Verification of result


13 attributes
Dataset
Preprocessing of data
Classification using SVM
Classification using logistic regression
Disease prediction


Heart Diseases Dataset
pre-processing step
Clean text
Remove Null data
Resample dataset
Random splits of data
Testing set (30%)
Training set (70%)
Cross Validation
Hyperparameters Optimization
Machine Learning Models
Performance Evaluation
Classification step
Predictions
Absence of heart disease
Presence of heart disease

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

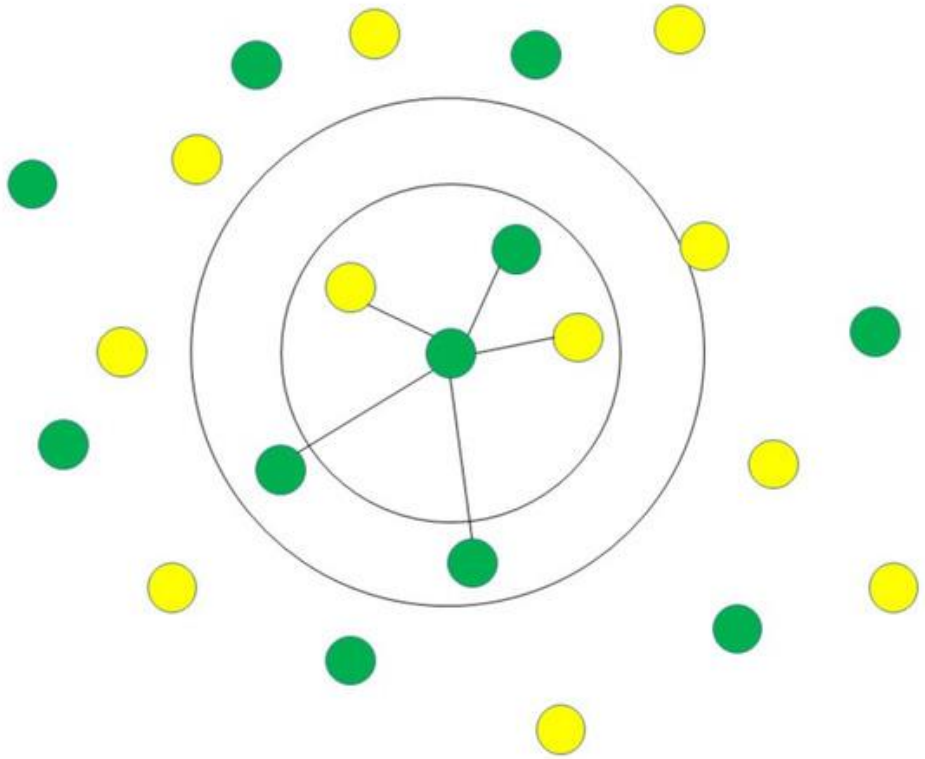
[redacted]

[redacted]

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

Author(s)	Technique	Specification	Limitation	Dataset	Result
Liu et al. (2025)	Systematic Review and Meta-analysis	Evaluates ML models using EHR across studies	Heterogeneity among EHR sources	Multiple EHR studies	AUC ≈ 91.6%
Amirahmadi et al. (2023)	Deep Learning on EHR Trajectories	Chronological EHR sequence-based DL models	Need for standard EHR trajectory models	MIMIC-III	Accuracy = 94.1%
Li et al. (2024)	ML on Irregular EHR Data	Handles irregular time gaps in EHRs for prediction	Challenge with data sparsity	Real-world EHR (Europe)	AUC = 90.3%
Houssein et al. (2023)	NLP & Deep Learning on EHR	Extracts features from unstructured EHR text	Language ambiguity and noise in text	EHR (Scientific Reports)	Accuracy = 92.8%
Ghasemieh et al. (2023)	Stacking Ensemble Learning	Predicts heart-disease readmission	Not validated on multi-institutional data	Heart-disease patient records	AUC = 93.4%
Ramakrishnaiah et al. (2025)	Data-driven ML Framework for EHR	Framework for ML pipeline using EHR	Need further real-time deployment testing	EHR from hospital systems	Not Quantified
Rahim et al. (2021)	Integrated ML Framework	Multi-model ML-based CVD detection	Generalizability untested	Open-access CVD datasets	Accuracy = 91.7%
Almulihi et al. (2022)	Hybrid Deep Learning Ensemble	Fusion of CNN + RNN with voting ensemble	Computationally intensive	UCI Cleveland + others	Accuracy = 94.7%
Mondal et al. (2024)	Dual-Stage Stacked ML	Combines stacking + RF + boosting models	Interpretability limited	Clinical datasets	Accuracy = 93.6%
Ogunpola et al. (2024)	Predictive ML Models	Applies XGBoost/Random Forest	Lack of explainability	Hospital cardiovascular records	Accuracy = 90.8%
Zhang et al. (2021)	Ensemble ML on Risk Factors + Echo	XGB + SVM + RF ensemble from EHR + Echo	Performance varies by input modality	CHD screening EHR + Echo	Accuracy = 95.1%

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
[ ]  # Install requirements

     !pip -q install kaggle shap imbalanced-learn seaborn matplotlib
     !pip -q install --upgrade scikit-learn
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.5/9.5 MB 64.4 MB/s eta 0:00:00
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
import os, json
from pathlib import Path


kaggle_token = {
    "username": "your_username",
    "key": "your_kaggle_api_key"
}

os.makedirs("/root/.kaggle", exist_ok=True)
with open("/root/.kaggle/kaggle.json", "w") as f:
    json.dump(kaggle_token, f)
!chmod 600 /root/.kaggle/kaggle.json

# Download Heart Failure Prediction dataset
!kaggle datasets download -d fedesoriano/heart-failure-prediction -p ./kdata

# Unzip
unzip_dir = Path("work/datasets/heart_failure")
unzip_dir.mkdir(parents=True, exist_ok=True)
!unzip -o -q ./kdata/heart-failure-prediction.zip -d {unzip_dir}

print("Files inside:", list(unzip_dir.iterdir()))
```

```python
import matplotlib.pyplot as plt
import seaborn as sns

num_cols = ['Age','RestingBP','Cholesterol','FastingBS','MaxHR','Oldpeak']
cat_cols = ['Sex','ChestPainType','RestingECG','ExerciseAngina','ST_Slope']

# numeric histograms
plt.figure(figsize=(12,10))
for i,col in enumerate(num_cols,1):
    plt.subplot(3,2,i)
    sns.histplot(df[col], kde=True)
    plt.title(col)
plt.tight_layout()
plt.show()
```

[redacted]

[redacted]

[redacted]

```python
# categorical counts
plt.figure(figsize=(12,10))
for i,col in enumerate(cat_cols,1):
    plt.subplot(3,2,i)
    sns.countplot(x=df[col])
    plt.title(col)
plt.tight_layout()
plt.show()
```

[redacted]

[redacted]

[redacted]

[redacted]

```
[ ]  # Correlation heatmap

     corr = df[num_cols+['HeartDisease']].corr()
     plt.figure(figsize=(8,6))
     sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
     plt.title("Correlation heatmap")
     plt.show()
```

[redacted]

[redacted]

[redacted]

```
[ ]  # Target balance

     df['HeartDisease'].value_counts().plot(kind='bar')
     plt.title("Heart Disease Distribution")
     plt.show()
     print(df['HeartDisease'].value_counts(normalize=True))
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

X = df.drop("HeartDisease", axis=1)
y = df["HeartDisease"]

numeric_features = ['Age','RestingBP','Cholesterol','FastingBS','MaxHR','Oldpeak']
categorical_features = ['Sex','ChestPainType','RestingECG','ExerciseAngina','ST_Slope']

preprocessor = ColumnTransformer([
    ("num", StandardScaler(), numeric_features),
    ("cat", OneHotEncoder(drop="first"), categorical_features)
])

X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=42,stratify=y)
```

```
[ ]  # Train multiple models

     from sklearn.linear_model import LogisticRegression
     from sklearn.svm import SVC
     from sklearn.ensemble import RandomForestClassifier

     models = {
         "LogisticRegression": LogisticRegression(max_iter=500),
         "SVM": SVC(probability=True, kernel="rbf"),
         "RandomForest": RandomForestClassifier(n_estimators=200, random_state=42)
     }

     fitted = {}
     for name, model in models.items():
         pipe = Pipeline([("preprocess", preprocessor), ("clf", model)])
         pipe.fit(X_train, y_train)
         fitted[name] = pipe
         print(f"{name} trained successfully.")
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
[ ] # Train multiple models

    from sklearn.linear_model import LogisticRegression
    from sklearn.svm import SVC
    from sklearn.ensemble import RandomForestClassifier

    models = {
        "LogisticRegression": LogisticRegression(max_iter=500),
        "SVM": SVC(probability=True, kernel="rbf"),
        "RandomForest": RandomForestClassifier(n_estimators=200, random_state=42)
    }

    fitted = {}
    for name, model in models.items():
        pipe = Pipeline([("preprocess", preprocessor), ("clf", model)])
        pipe.fit(X_train, y_train)
        fitted[name] = pipe
        print(f"{name} trained successfully.")
```

[redacted]

[redacted]

[redacted]

[redacted]

```
[ ] # Evaluate Models

    from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix, RocCurveDisplay, PrecisionRecallDisplay

    metrics = {}
    for name, pipe in fitted.items():
        y_pred = pipe.predict(X_test)
        y_prob = pipe.predict_proba(X_test)[:,1]
        metrics[name] = {
            "Accuracy": accuracy_score(y_test,y_pred),
            "Precision": precision_score(y_test,y_pred),
            "Recall": recall_score(y_test,y_pred),
            "F1": f1_score(y_test,y_pred),
            "ROC-AUC": roc_auc_score(y_test,y_prob)
        }
    pd.DataFrame(metrics).T
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
# SHAP Explainability

import shap
import numpy as np

best_model_name = max(metrics, key=lambda k: metrics[k]['ROC-AUC'])
best_pipe = fitted[best_model_name]
print("Best model:", best_model_name)

# Transform features
X_train_transformed = best_pipe.named_steps['preprocess'].fit_transform(X_train)
X_test_transformed = best_pipe.named_steps['preprocess'].transform(X_test)
feature_names = best_pipe.named_steps['preprocess'].get_feature_names_out()
clf = best_pipe.named_steps['clf']
```

[redacted]

[redacted]

[redacted]

```python
if best_model_name == "SVM":
    # KernelExplainer expects a scalar output, so we pass only class-1 probability
    f = lambda x: clf.predict_proba(x)[:, 1]

    # Background dataset (small for speed)
    background = shap.sample(X_train_transformed, 50, random_state=42)

    explainer = shap.KernelExplainer(f, background)

    # Explain 10 test points
    shap_values = explainer.shap_values(X_test_transformed[:10])

    print("shap_values shape:", np.array(shap_values).shape,
          "| features shape:", X_test_transformed[:10].shape)

    shap.summary_plot(shap_values, X_test_transformed[:10], feature_names=feature_names)
```

[redacted]

[redacted]

```
elif best_model_name == "RandomForest":
    explainer = shap.TreeExplainer(clf)
    shap_values = explainer.shap_values(X_train_transformed[:200])
    shap.summary_plot(shap_values[1], X_train_transformed[:200], feature_names=feature_names)

elif best_model_name == "LogisticRegression":
    explainer = shap.LinearExplainer(clf, X_train_transformed, feature_dependence="independent")
    shap_values = explainer.shap_values(X_train_transformed[:200])
    shap.summary_plot(shap_values, X_train_transformed[:200], feature_names=feature_names)
```

```python
# SHAP Waterfall Plot

import matplotlib.pyplot as plt

if best_model_name == "SVM":

    idx = 0
    shap_value_single = explainer.shap_values(X_test_transformed[idx:idx+1])

    # Ensure shape is correct (convert to array)
    shap_value_single = np.array(shap_value_single).reshape(1, -1)

    shap.waterfall_plot(
        shap.Explanation(values=shap_value_single[0],
                         base_values=explainer.expected_value,
                         data=X_test_transformed[idx],
                         feature_names=feature_names)
    )
    plt.show()

elif best_model_name == "RandomForest":
    shap_values_single = explainer.shap_values(X_test_transformed[idx:idx+1])
    shap.waterfall_plot(
        shap.Explanation(values=shap_values_single[1][0],
                         base_values=explainer.expected_value[1],
                         data=X_test_transformed[idx],
                         feature_names=feature_names)
    )
    plt.show()

elif best_model_name == "LogisticRegression":
    shap_values_single = explainer.shap_values(X_test_transformed[idx:idx+1])
    shap.waterfall_plot(
        shap.Explanation(values=shap_values_single[0],
                         base_values=explainer.expected_value,
                         data=X_test_transformed[idx],
                         feature_names=feature_names)
    )
```

[redacted]

[redacted]

[redacted]

```
[ ]
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

best_model_name = max(metrics, key=lambda k: metrics[k]['ROC-AUC'])
best_pipe = fitted[best_model_name]
print("Best model:", best_model_name)

# Get feature names after preprocessing
feature_names = best_pipe.named_steps['preprocess'].get_feature_names_out()
clf = best_pipe.named_steps['clf']

# Different methods depending on model type
if best_model_name == "RandomForest":
    importances = clf.feature_importances_
elif best_model_name == "LogisticRegression":
    importances = np.abs(clf.coef_[0])
elif best_model_name == "SVM":


    from sklearn.inspection import permutation_importance
    X_test_trans = best_pipe.named_steps['preprocess'].transform(X_test)
    result = permutation_importance(clf, X_test_trans, y_test, n_repeats=20, random_state=42, n_jobs=-1)
    importances = result.importances_mean

# Create dataframe
feat_imp = pd.DataFrame({
    "Feature": feature_names,
    "Importance": importances
}).sort_values(by="Importance", ascending=False)

# Plot
plt.figure(figsize=(10,6))
sns.barplot(x="Importance", y="Feature", data=feat_imp)
plt.title(f"Feature Importance — {best_model_name}")
plt.show()
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

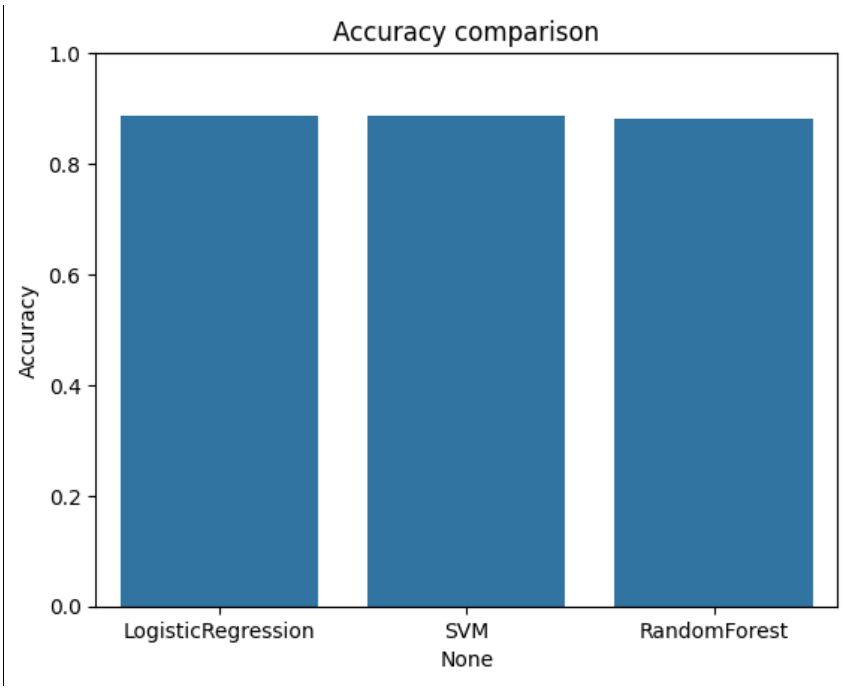
[redacted]

[redacted]

[redacted]

	Accuracy	Precision	Recall	F1	ROC-AUC
LogisticRegression	0.885870	0.871560	0.931373	0.900474	0.930775
SVM	0.885870	0.885714	0.911765	0.898551	0.946437

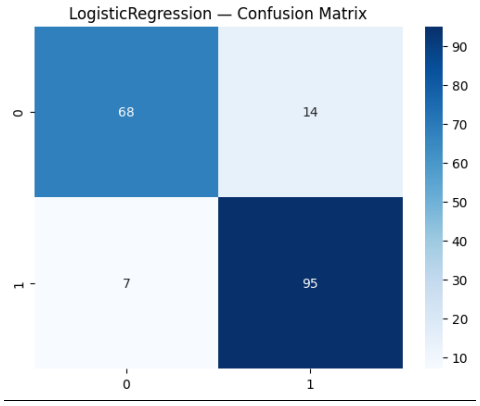
RandomForest	0.880435	0.892157	0.892157	0.892157	0.929758



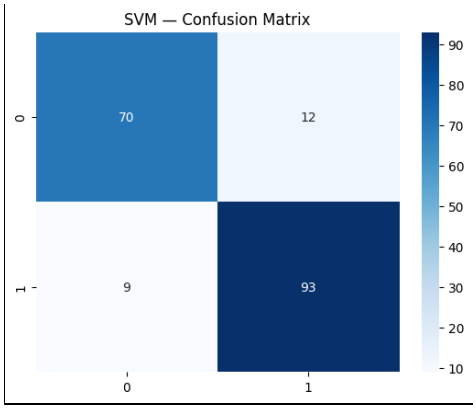
[redacted]

[redacted]

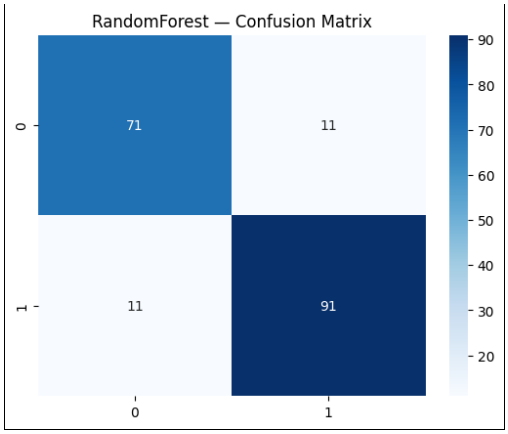
[redacted]



[redacted]



[redacted]



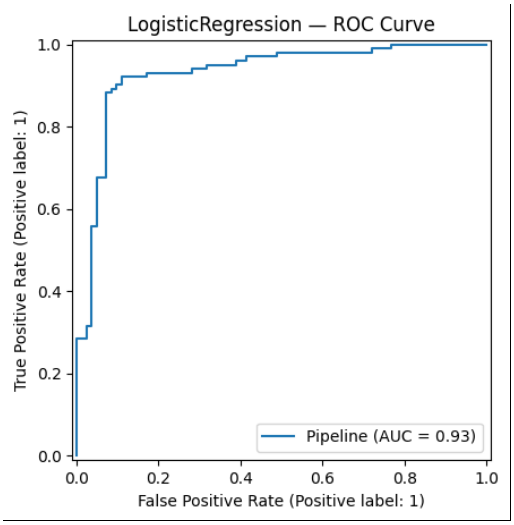
[redacted]

[redacted]

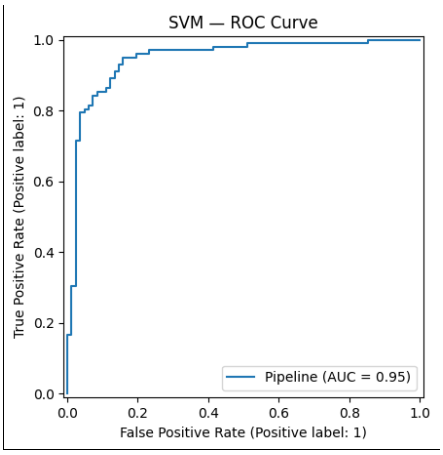
[redacted]

[redacted]

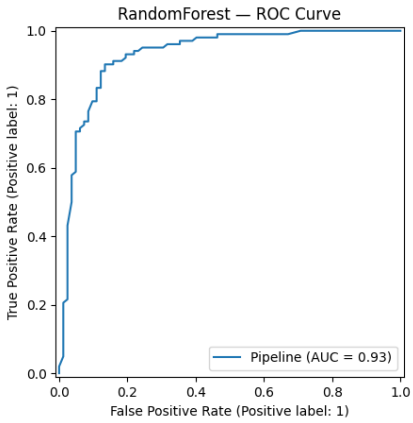
[redacted]



[redacted]



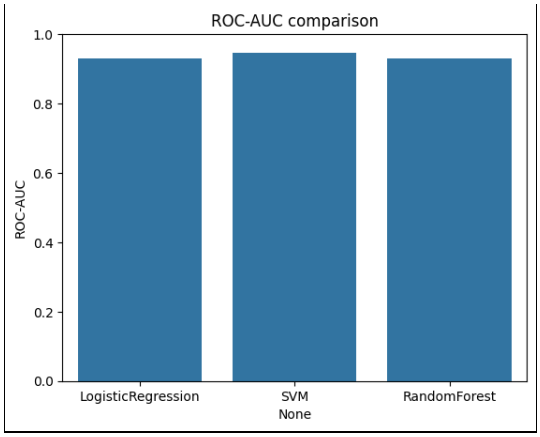
[redacted]



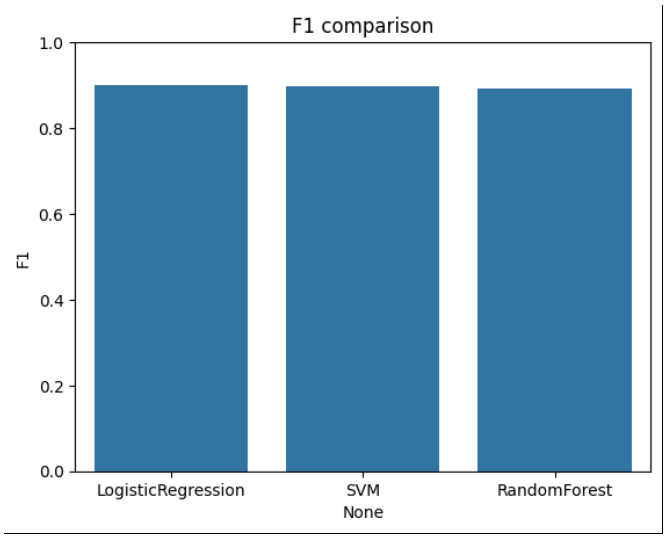
[redacted]

[redacted]

[redacted]



[redacted]



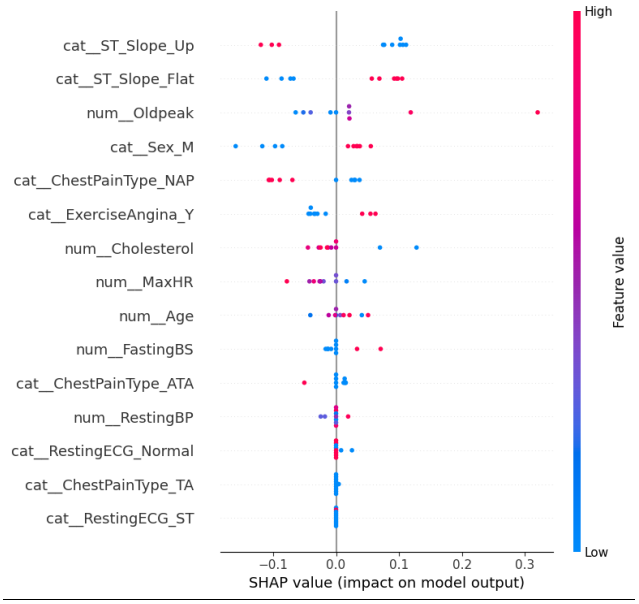
[redacted]

[redacted]

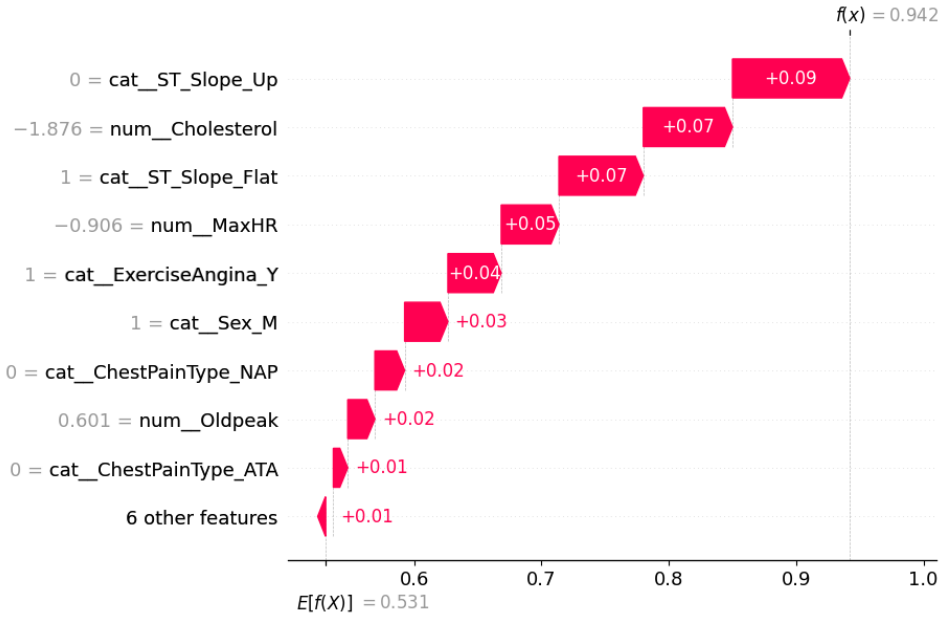
[redacted]

[redacted]

[redacted]


cat__ST_Slope_Up
cat__ST_Slope_Flat
num__Oldpeak
cat__Sex_M
cat__ChestPainType_NAP
cat__ExerciseAngina_Y
num__Cholesterol
num__MaxHR
num__Age
num__FastingBS
cat__ChestPainType_ATA
num__RestingBP
cat__RestingECG_Normal
cat__ChestPainType_TA
cat__RestingECG_ST
−0.1
0.0
0.1
0.2
0.3
SHAP value (impact on model output)
High
Feature value
Low

[redacted]



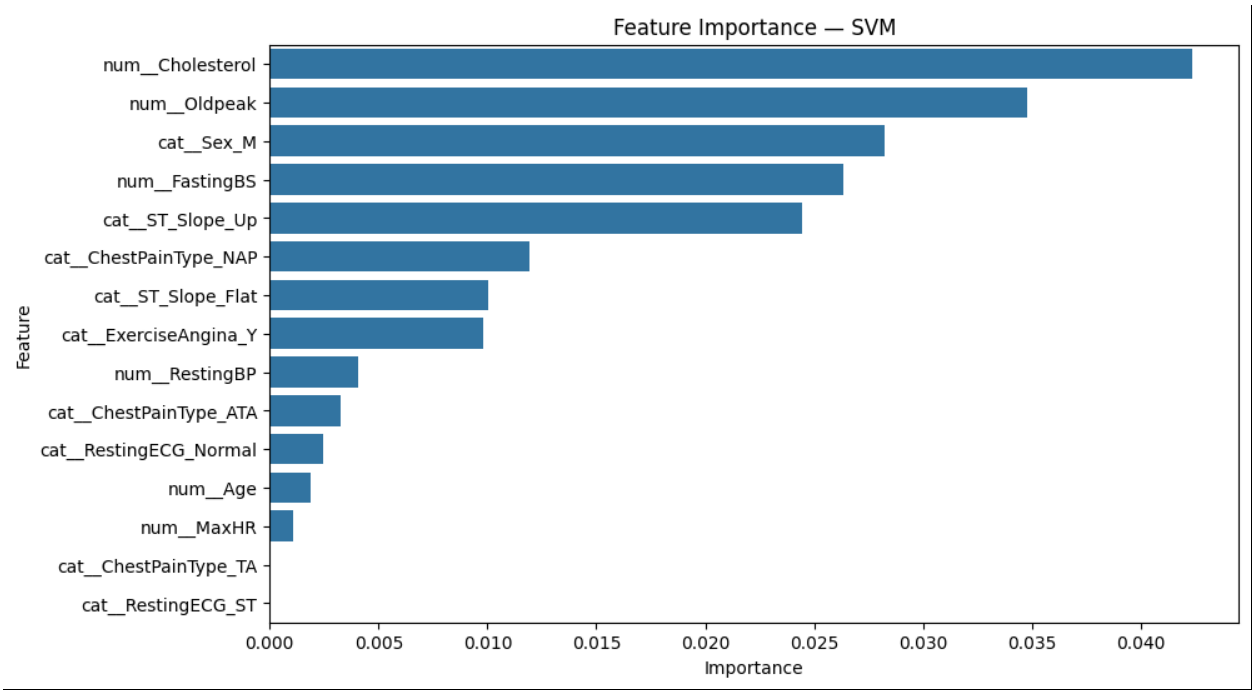
[redacted]

[redacted]

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]